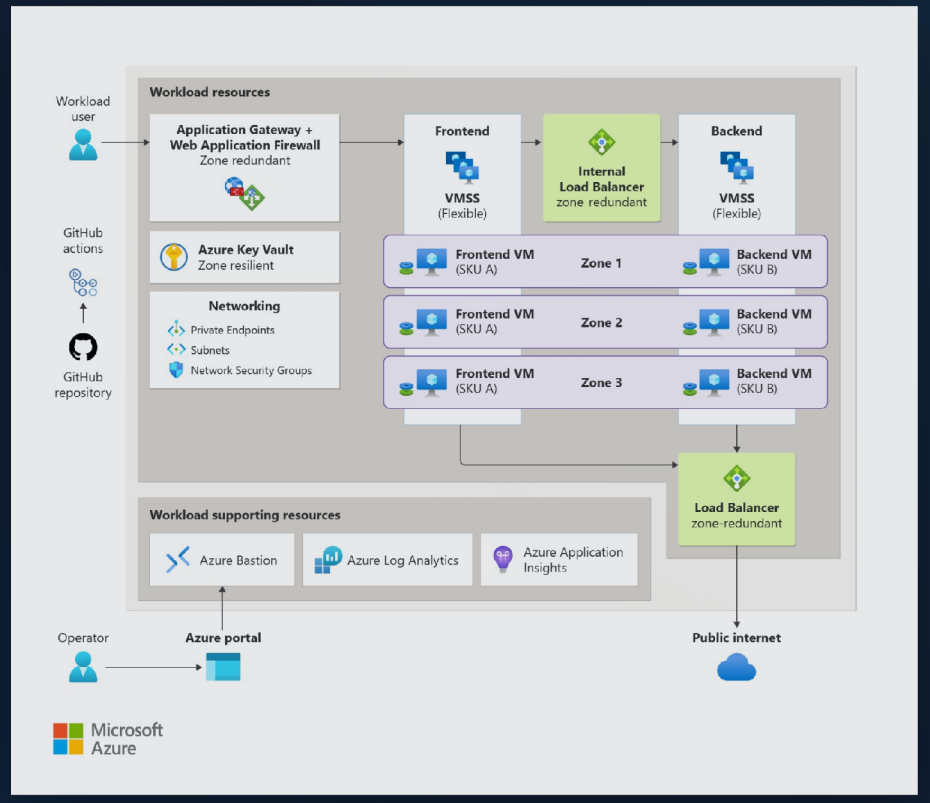
Burger Builder DevOps Platform on Azure

From application repo to automated delivery platform. React frontend & Spring Boot backend deployed as containers.

Terraform | Ansible | GitHub Actions | Secure HTTPS



Current repository state, not idealized future state

Project Goal & Scope

Delivery	Application Gateway + HTTPS
Security	Key Vault + Managed Identities
CI/CD	Build, Test, Deploy Automation
Configuration	Ansible Host Setup + Docker
Infrastructure	Terraform Provisioning

- Deploy **frontend, backend, database** connectivity, and SonarQube
- Provision Azure infrastructure through **Terraform**
- Configure hosts and deploy containers through **Ansible**
- Build, test, scan, and deploy through **GitHub Actions**
- Expose the platform through Application Gateway with **HTTPS**

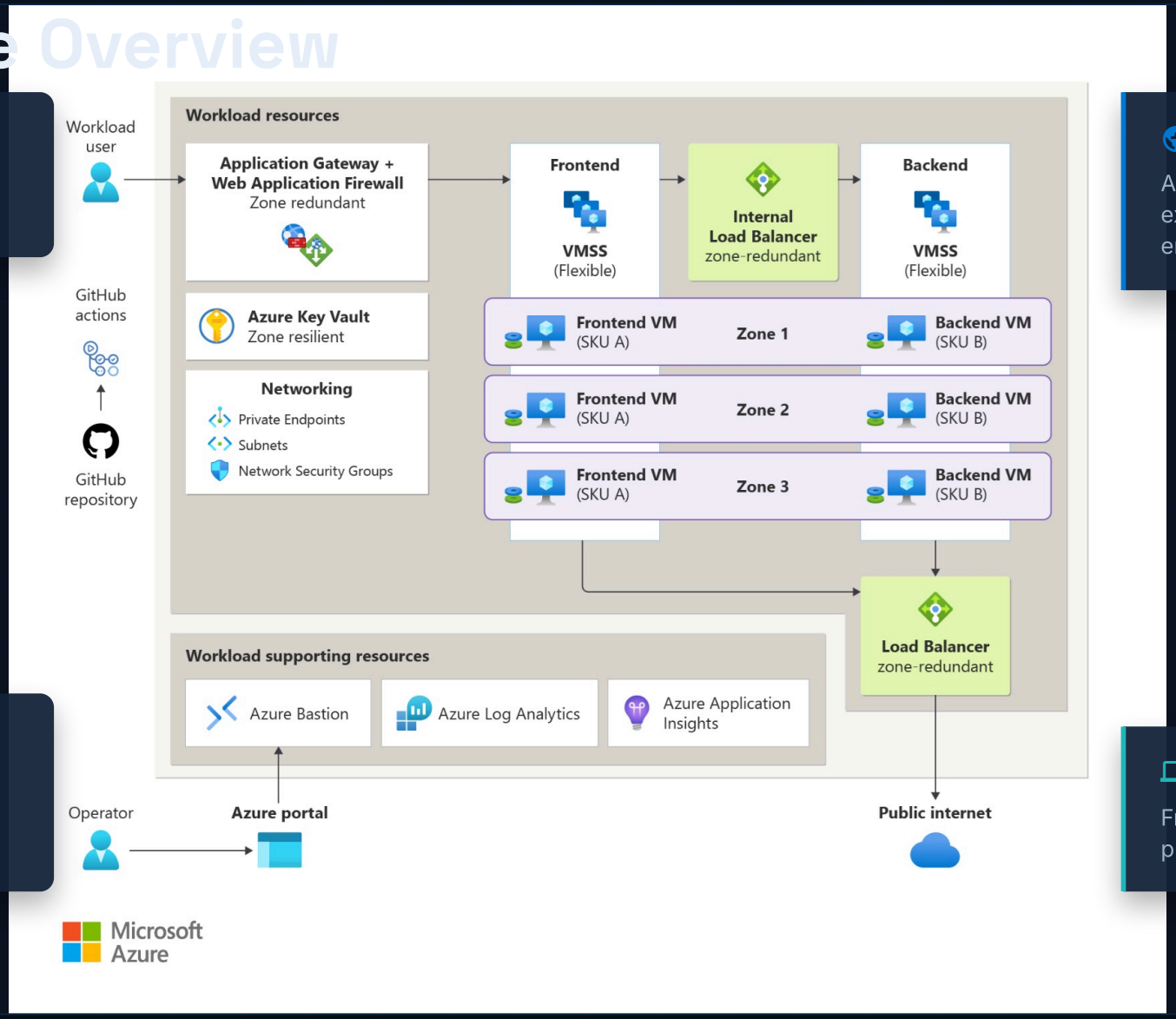
Architecture Overview

Ops Tier

Ops VM hosts SonarQube and internal automation runner.

Public Edge

Application Gateway handles external traffic. Only public endpoint for app access.



Data Tier

Azure SQL secured via Private Endpoint. No public access enabled.

Private App Tier

Frontend & Backend VMs reside on private IPs within VNet subnets.

Infrastructure As Code With Terraform

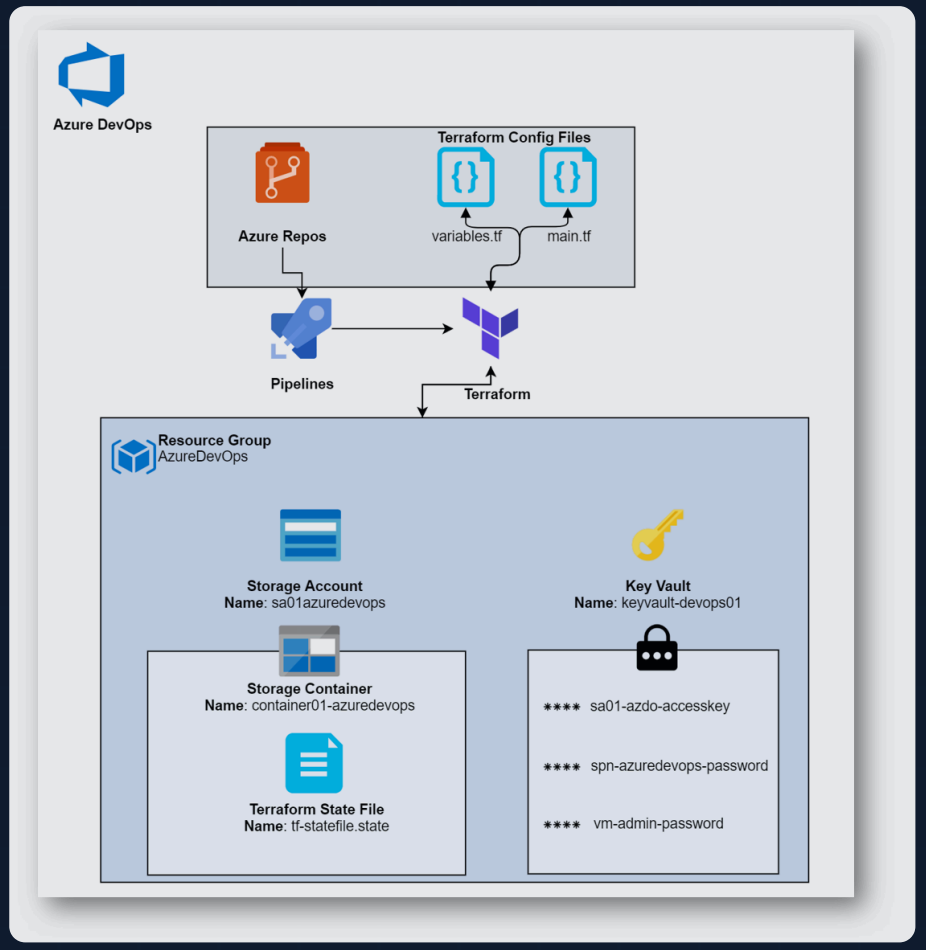
- Networking
- Virtual Machines
- Azure SQL
- Monitoring
- Key Vault
- App Gateway

Key Outputs

- Host IPs
- ACR Details
- Key Vault Name
- Subnet CIDRs
- Frontend Origin

Single source of infrastructure truth

Terraform Control Plane



Configuration And Deployment With Ansible

Deployment Automation Flow

Terraform Output -json
Dynamic inventory source

↓

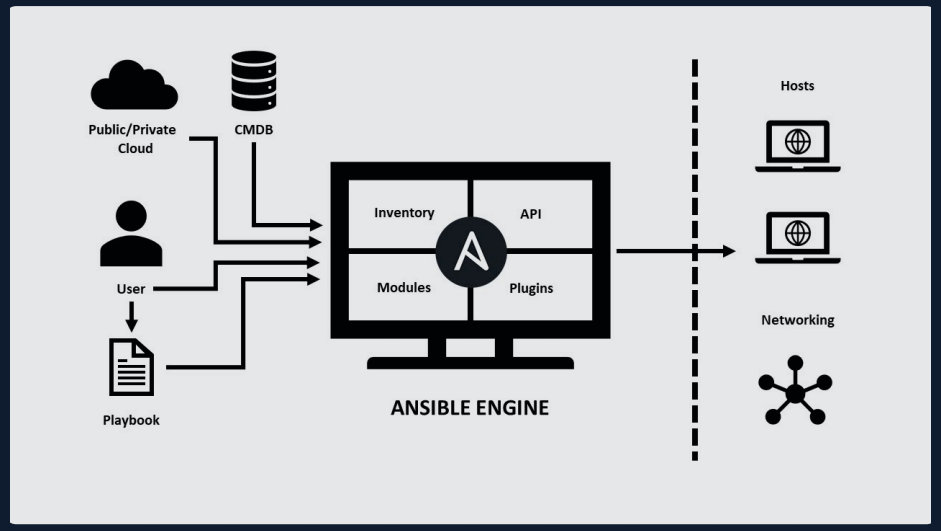
Ansible Dynamic Inventory
site.yml builds in-memory hosts

↓

Role Execution
Common, Docker, Frontend, Backend, SonarQube

No Static Inventory Files
Terraform is the single source of truth for host definitions

ANSIBLE ENGINE ARCHITECTURE



Key Features

- VMs use **managed identity** for Azure and ACR access
- Backend and SonarQube fetch secrets from **Key Vault**
- Deployment tied directly to **infrastructure state**

CI/CD Pipeline Design

backend.yml
Verify, Scan, Build, Push, Deploy
GitHub Hosted | Self-Hosted

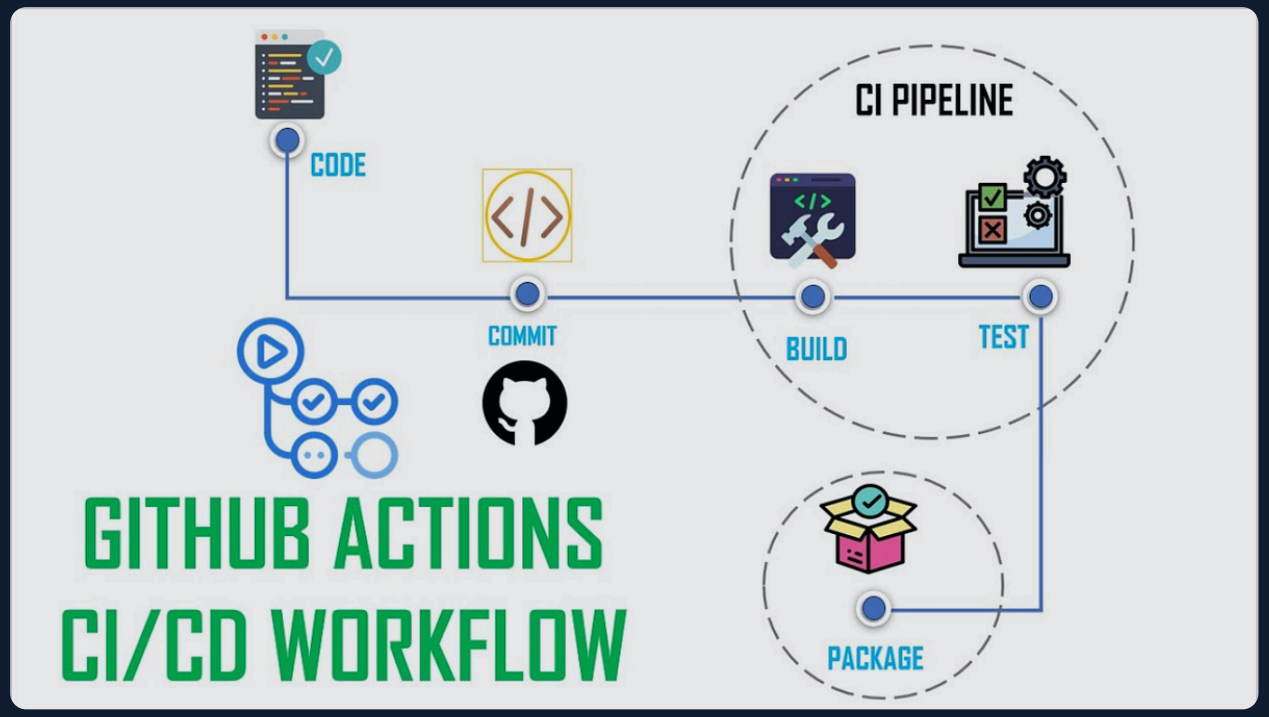
frontend.yml
Verify, Scan, Build, Push, Deploy
GitHub Hosted | Self-Hosted

infra.yml
Terraform Format, Init, Validate, Plan, Apply
GitHub Hosted | Self-Hosted

iac-validation.yml
PR IaC Checks
GitHub Hosted

appgw-certificate.yml
Certificate Issue/Renew
Self-Hosted

Workflow Architecture



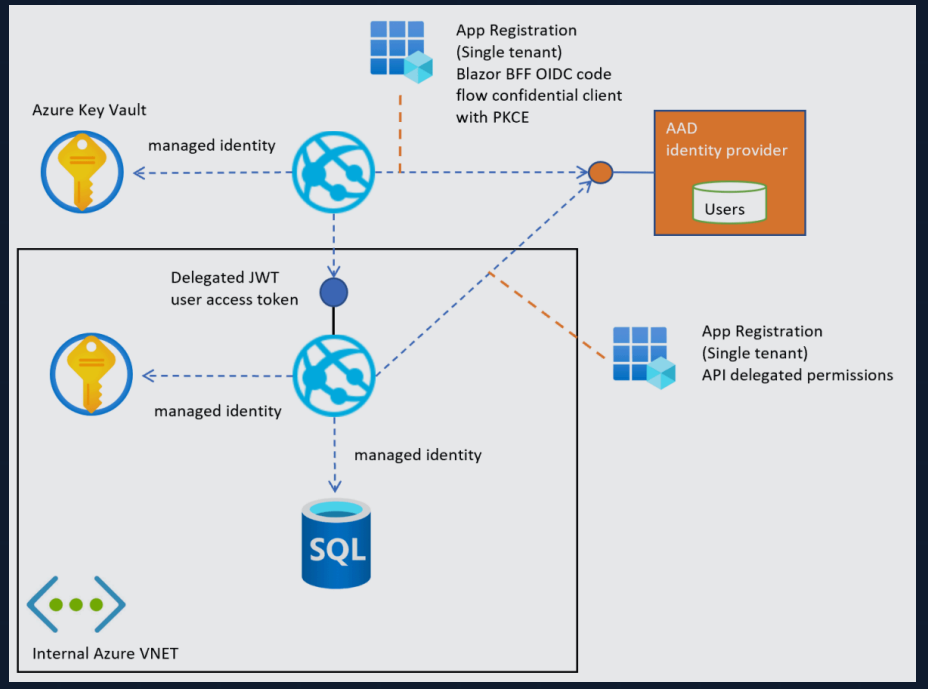
Security Design

Identity

Frontend, Backend, and Ops VMs use System-Assigned Managed Identities

Secrets

Key Vault policies separate secret reading from certificate management



Registry

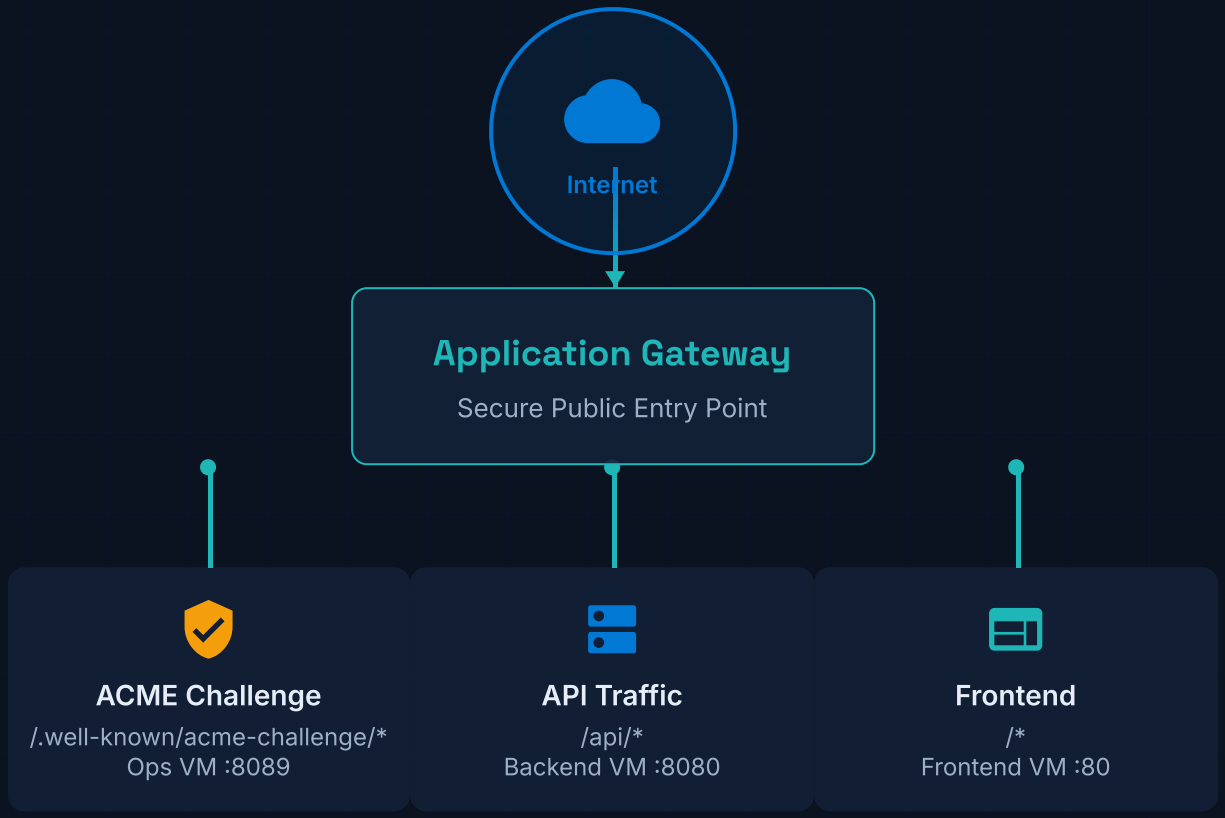
AcrPull for Frontend, Backend
AcrPush for Ops VM

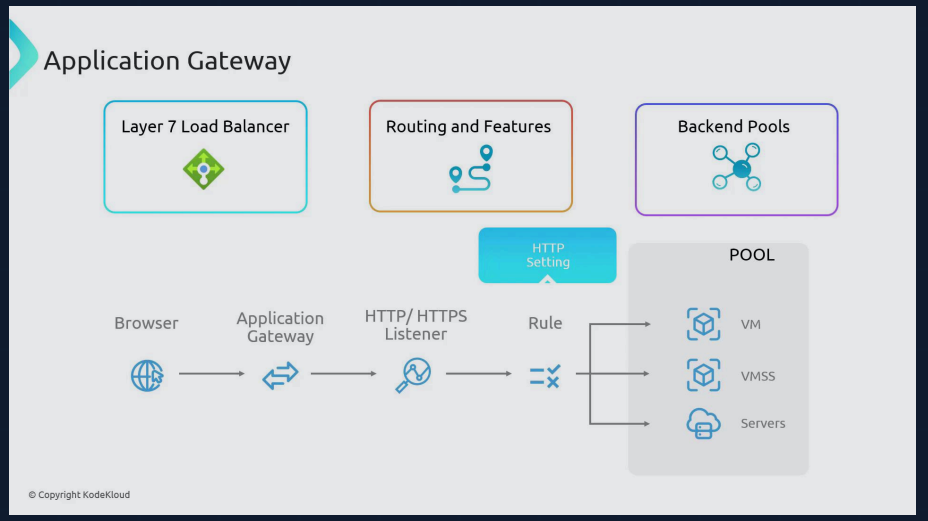
Private Data

Azure SQL disables public access, stays behind private endpoint

Minimize static credentials

Traffic Flow And HTTPS





HTTPS Redirection

HTTP permanently redirects to HTTPS when enabled

Default Hostname

Uses generated **sslip.io** hostname based on public IP

Automated Certificate Issuance

01

GitHub Workflow

appgw-certificate.yml runs on self-hosted ops runner

02

ACME Challenge

Start temporary webroot server on port 8089

03

Let's Encrypt

Issues certificate for Application Gateway hostname

04

Key Vault Import

Convert to PFX and import into Azure Key Vault

05

HTTPS Enabled

Terraform applies config to attach certificate to Gateway

Code Complete

Session execution pending

Implementation Status

- Fully designed and coded automation sequence
- Uses sslip.io for automated hostname generation
- Browser-trusted TLS ready to deploy

Problems Found And Fixed

01

Issue
Dynamic inventory bootstrap failed with --limit

Fix
Corrected inventory construction logic

02

Issue
ProxyJump logic broken for self-hosted runner inside VNet

Fix
Corrected SSH pathing for internal routing

03

Issue
No backend health probe for Application Gateway

Fix
Added health probe at /api/health

04

Issue
CI mixed verification and deployment in one stage

Fix
Split into distinct verification and deployment stages

05

Issue
Static credentials for Terraform Azure authentication

Fix
Added Azure OIDC authentication to workflow

06

Issue
Manual HTTPS and certificate management

Fix
Automated certificate issuance and HTTPS configuration

Validation Status

Confirmed

- Static repo review completed
- Ansible syntax-check succeeded

Blocked by Environment

- Terraform validation blocked (provider download restricted)

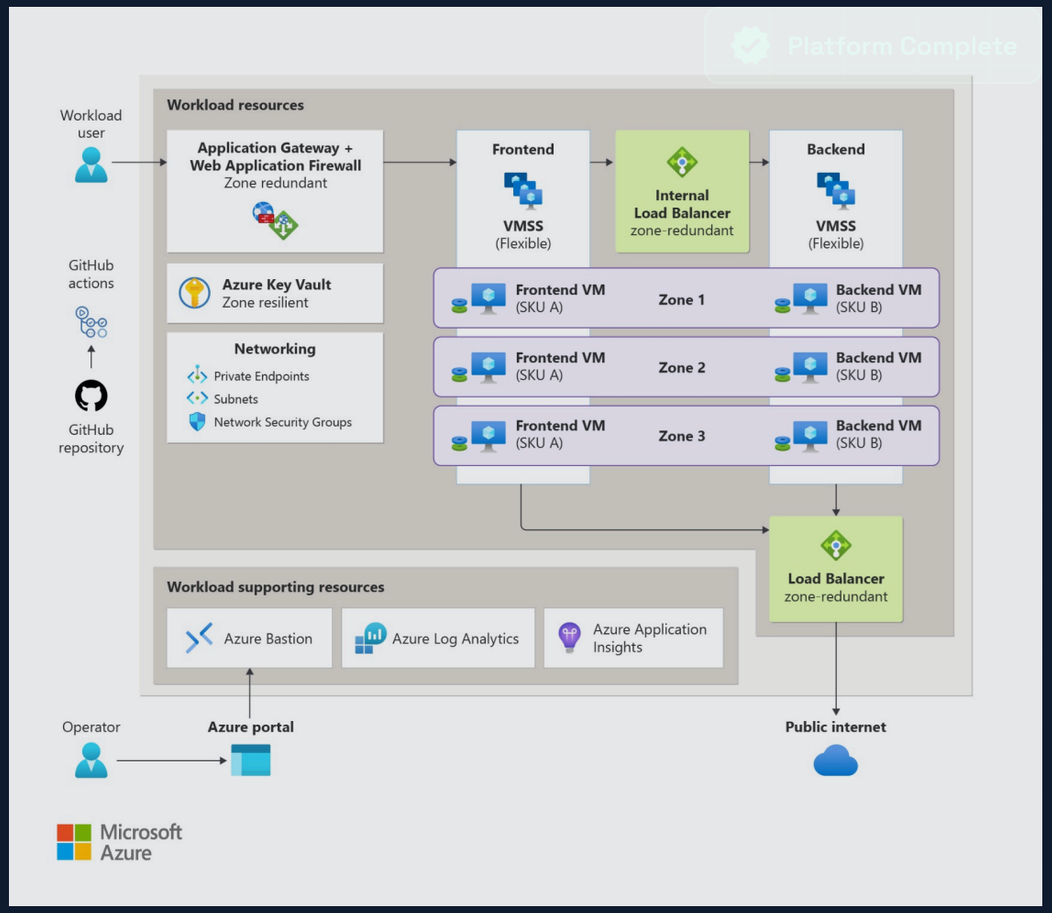
Not Executed in Session

- No end-to-end apply, deploy, or certificate issuance run

Present current evidence, not assumptions

Conclusion And Next Steps

- Infrastructure, deployment, secrets, and HTTPS automation connected into one platform
- Repository is materially closer to production-style delivery than starting baseline
- Clear infrastructure ownership and dynamic configuration



Next Milestone

Run the GitHub workflows in Azure and verify the full path end to end

Future Improvements